

AWS IAM Role for EC2 to S3 Uploads

To Begin with the Lab

Summary of the Lab

In this lab, the difference between the insecure access-key approach and the recommended **AWS IAM role** approach was demonstrated using an **Amazon EC2** web application that uploads files to **Amazon S3**. First, an IAM user was created and given S3 upload permissions through a customer-managed policy, and that user's access key and secret access key were embedded directly into the Flask application running on EC2. The application worked and successfully uploaded a file to the S3 bucket, but the method was shown to be unsafe because the credentials were stored in code. Next, the insecure setup was replaced with an **IAM role** attached to the EC2 instance, and the application was updated to use Boto3 without hardcoded AWS credentials. The same upload flow then worked again, but this time using temporary role credentials, which matched AWS best practices.

Understanding IAM Roles for EC2 Access

An **IAM role** is a temporary identity that AWS services and users can assume when they need permission to do a job. It exists so you do not have to store long-term credentials inside an application or server. In this lab, the EC2 instance needed permission to upload files to S3, and the role gave it temporary credentials automatically. That is safer than using an IAM user's access key and secret key because the application can keep working without exposing secrets in code.

Example:

A Flask app running on an EC2 instance accepts a photo upload and sends it to an S3 bucket named myapp03. In the insecure version, the app contains an access key and secret key for S3AccessUser. In the secure version, the EC2 instance assumes MyAppS3Access as an IAM role and uploads the file without any AWS keys stored in the application.

Lab Steps

Step 1 – Create the S3 Bucket

- Sign in to the AWS Management Console.
- Open **Amazon S3**.
- Click **Create bucket**.

Amazon S3 > Buckets

Buckets

General purpose buckets | All AWS Regions | Directory buckets

General purpose buckets (225) Info Copy ARN Empty Delete **Create bucket**

Buckets are containers for data stored in S3.

Find buckets by name

	Name	AWS Region	Creation date
☐	...	Asia Pacific (Mumbai) ap-south-1	February 12, 2026, 00:44:57 (UTC+05:30)
☐	...	US East (N. Virginia) us-east-1	June 16, 2026, 15:49:49 (UTC+05:30)
☐	...	Asia Pacific (Mumbai) ap-south-1	June 16, 2026, 15:48:52 (UTC+05:30)
☐	...	Asia Pacific (Mumbai) ap-south-1	November 13, 2025, 15:18:01 (UTC+05:30)
☐	...	Asia Pacific (Mumbai) ap-south-1	January 22, 2026, 11:32:08 (UTC+05:30)
☐	...	Asia Pacific (Mumbai) ap-south-1	November 12, 2025, 11:03:13 (UTC+05:30)
☐	...	Asia Pacific (Mumbai) ap-south-1	November 4, 2025, 18:41:42 (UTC+05:30)
☐	...	Asia Pacific (Mumbai) ap-south-1	December 27, 2025, 11:32:40 (UTC+05:30)
☐	...	Asia Pacific (Mumbai) ap-south-1	May 22, 2026, 20:59:53 (UTC+05:30)
☐	...	Asia Pacific (Mumbai) ap-south-1	April 19, 2026, 14:47:46 (UTC+05:30)

Account snapshot Info View dashboard
Updated daily
Storage Lens provides visibility into storage usage and activity trends.

External access summary Info
Updated daily
External access findings help you identify bucket permissions that allow public access or access from other AWS accounts.

- Enter the bucket name: my-app03
- Keep the bucket in the selected AWS Region.

Create bucket Info

Buckets are containers for data stored in S3.

General configuration

AWS Region
US East (N. Virginia) us-east-1

Bucket type Info

☒ **General purpose**
Recommended for most use cases and access patterns. General purpose buckets are the original S3 bucket type. They allow a mix of storage classes that redundantly store objects across multiple Availability Zones.

☐ **Directory**
Recommended for low-latency use cases. These buckets use only the S3 Express One Zone storage class, which provides faster processing of data within a single Availability Zone.

Bucket namespace
Choose the namespace where you want to create your bucket. [Learn more](#)

☒ **Global namespace**
By default, S3 creates general purpose buckets in the global namespace.

☐ **Account Regional namespace (recommended)**
General purpose buckets created in your account Regional namespace are unique to your account. These buckets can never be created by another AWS account.

Bucket name Info

my-app03

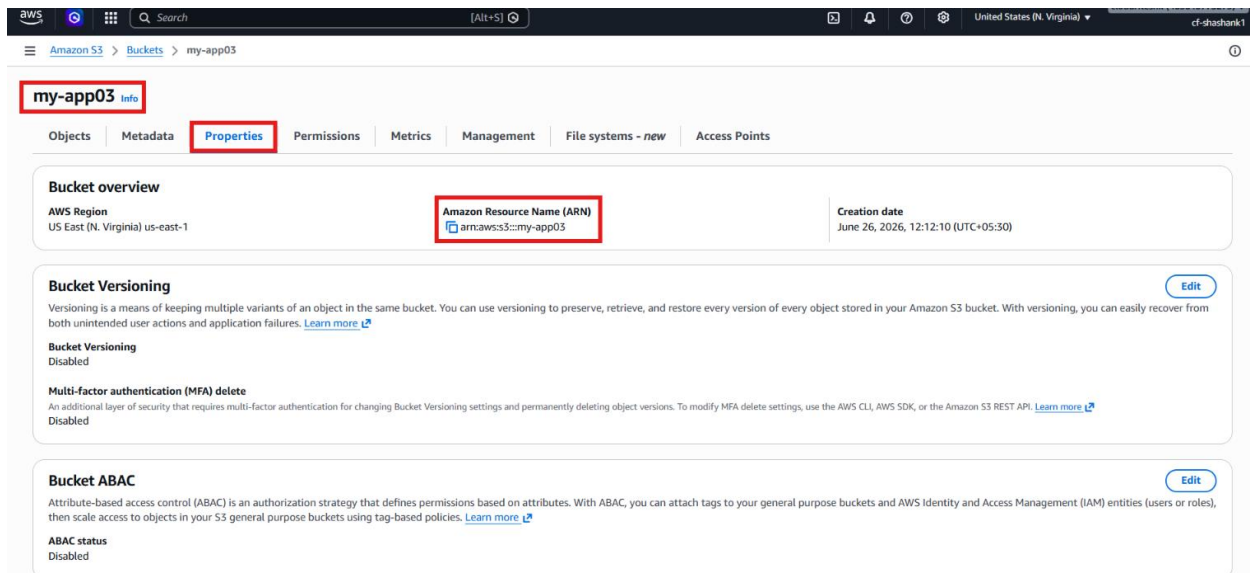
Bucket names must be 3 to 63 characters and unique within the global namespace. Bucket names must also begin and end with a letter or number. Valid characters are a-z, 0-9, periods (.), and hyphens (-). [Learn more](#)

Copy settings from existing bucket - optional
Only the bucket settings in the following configuration are copied.

[Choose bucket](#)

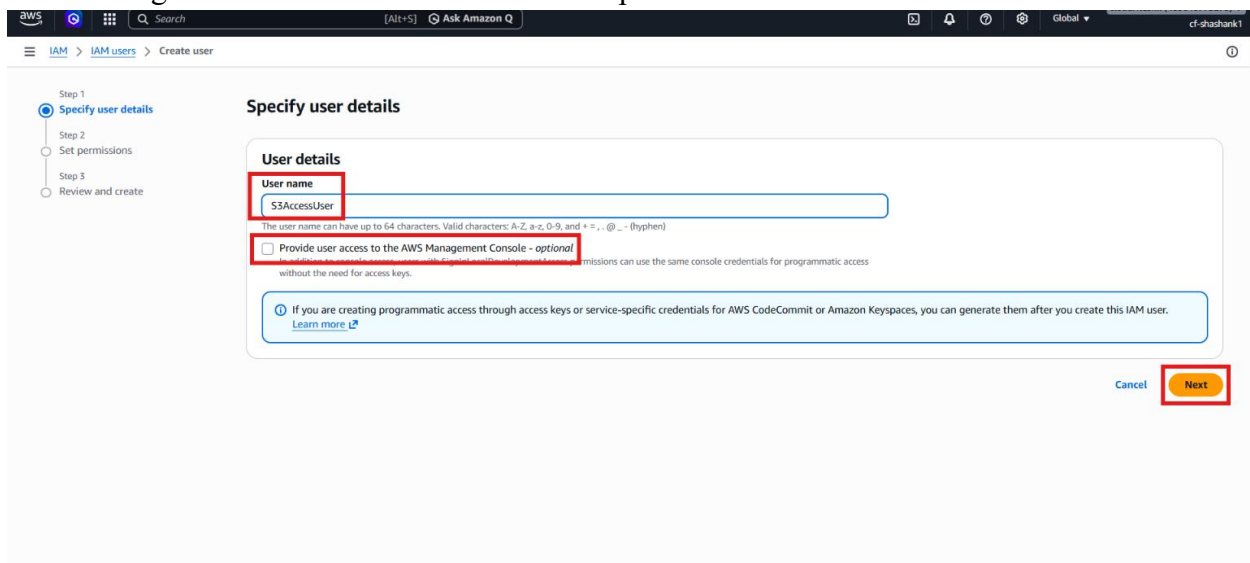
Format: s3://bucket/prefix

- Create the bucket.
- Open the bucket properties and note its ARN for the policy.



Step 2 – Create the IAM User for the Insecure Demo

- Open IAM.
- Click Users.
- Click Create user.
- Enter the username: S3AccessUser
- Do not give this user console access and no permission for now.



- Create the user.

Review and create

Review your choices. After you create the user, you can view and download the autogenerated password, if enabled.

User details

User name S3AccessUser	Console password type None	Require password reset No
----------------------------------	--------------------------------------	-------------------------------------

Permissions summary

Name	Type	Used as
No resources		

Tags - optional

Tags are key-value pairs you can add to AWS resources to help identify, organize, or search for resources. Choose any tags you want to associate with this user.

No tags associated with the resource.

[Add new tag](#)

You can add up to 50 more tags.

[Cancel](#) [Previous](#) [Create user](#)

- Open the user and click **Create access key**.

S3AccessUser

Summary

ARN arn:aws:iam::4[redacted]:user/S3AccessUser	Console access Disabled	Access key 1 Create access key
Created June 26, 2026, 12:19 (UTC+05:30)	Last console sign-in -	

Permissions | Groups | Tags | Security credentials | Last Accessed

Permissions policies (0)

Permissions are defined by policies attached to the user directly or through groups.

[Search](#) [Filter by Type](#) [All types](#)

Policy name	Type	Attached via
No resources to display		

Permissions boundary (not set)

Generate policy based on CloudTrail events

- Select the **Application running on AWS compute service** use case.
- Tick the confirmation.

Step 1: Retrieve access keys

Command Line Interface (CLI)
You plan to use this access key to enable the AWS CLI to access your AWS account.

Local code
You plan to use this access key to enable application code in a local development environment to access your AWS account.

Application running on an AWS compute service
You plan to use this access key to enable application code running on an AWS compute service like Amazon EC2, Amazon ECS, or AWS Lambda to access your AWS account.

Third-party service
You plan to use this access key to enable access for a third-party application or service that monitors or manages your AWS resources.

Application running outside AWS
You plan to use this access key to authenticate workloads running in your data center or other infrastructure outside of AWS that needs to access your AWS resources.

Other
Your use case is not listed here.

Alternative recommended
Assign an IAM role to compute resources like EC2 instances or Lambda functions to automatically supply temporary credentials to enable access.
[Learn more](#)

Confirmation
☒ I understand the above recommendation and want to proceed to create an access key.

[Cancel](#) **Next**

- Save the access key and secret access key.

This is the only time that the secret access key can be viewed or downloaded. You cannot recover it later. However, you can create a new access key any time.

Step 1: Access key best practices & alternatives
Step 2 - optional: Set description tag
Step 3: **Retrieve access keys**

Retrieve access keys [Info](#)

Access key
If you lose or forget your secret access key, you cannot retrieve it. Instead, create a new access key and make the old key inactive.

Access key: AKIAI44QH8DHBVS3GMA5E2 Secret access key: ***** [Show](#)

Access key best practices

- Never store your access key in plain text, in a code repository, or in code.
- Disable or delete access key when no longer needed.
- Enable least-privilege permissions.
- Rotate access keys regularly.

For more details about managing access keys, see the [best practices for managing AWS access keys](#).

[Download .csv file](#) **Done**

Step 3 – Create the S3 Upload Policy

- Open **IAM** → **Policies**.
- Click **Create policy**.
- Switch to the **JSON** tab.
- Paste the S3 upload policy.
- Replace the bucket placeholder with your bucket ARN.

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
```

```

"Effect": "Allow",
"Action": "s3:PutObject",
"Resource": [
    "<YOUR-BUCKET-ARN/>"
]
}
]
}

```

Specify permissions Info

Add permissions by selecting services, actions, resources, and conditions. Build permission statements using the JSON editor.

Step 1 **Specify permissions**
Step 2 Review and create

Policy editor

Visual **JSON** Actions

Edit statement

Select a statement

Select an existing statement in the policy or add a new statement.

[+ Add new statement](#)

```

1 {
2   "Version": "2012-10-17",
3   "Statement": [
4     {
5       "Effect": "Allow",
6       "Action": "s3:PutObject",
7       "Resource": [
8         "<YOUR-BUCKET-ARN/>"
9       ]
10    }
11  ]
12 }
13

```

- Name the policy: S3PutObjectPermission
- Create the policy.

Review and create Info

Review the permissions, specify details, and tags.

Step 1 Specify permissions
Step 2 **Review and create**

Policy details

Policy name

Enter a meaningful name to identify this policy.

S3PutObjectPermission

Maximum 128 characters. Use alphanumeric and "+, -, @, _" characters.

Description - optional

Add a short explanation for this policy.

Maximum 1,000 characters. Use alphanumeric and "+, -, @, _" characters.

ⓘ This policy defines some actions, resources, or conditions that do not provide permissions. To grant access, policies must have an action that has an applicable resource or condition. For details, choose [Show remaining](#). [Learn more](#)

Permissions defined in this policy Info

Permissions defined in this policy document specify which actions are allowed or denied. To define permissions for an IAM identity (user, user group, or role), attach a policy to it

Allow (1 of 471 services)

Services | Actions | Resources | Permissions

Show remaining 470 services

Step 4 – Attach the Policy to the IAM User

- Return to IAM → Users.

- Select **S3AccessUser**.
- Click **Add permissions**.

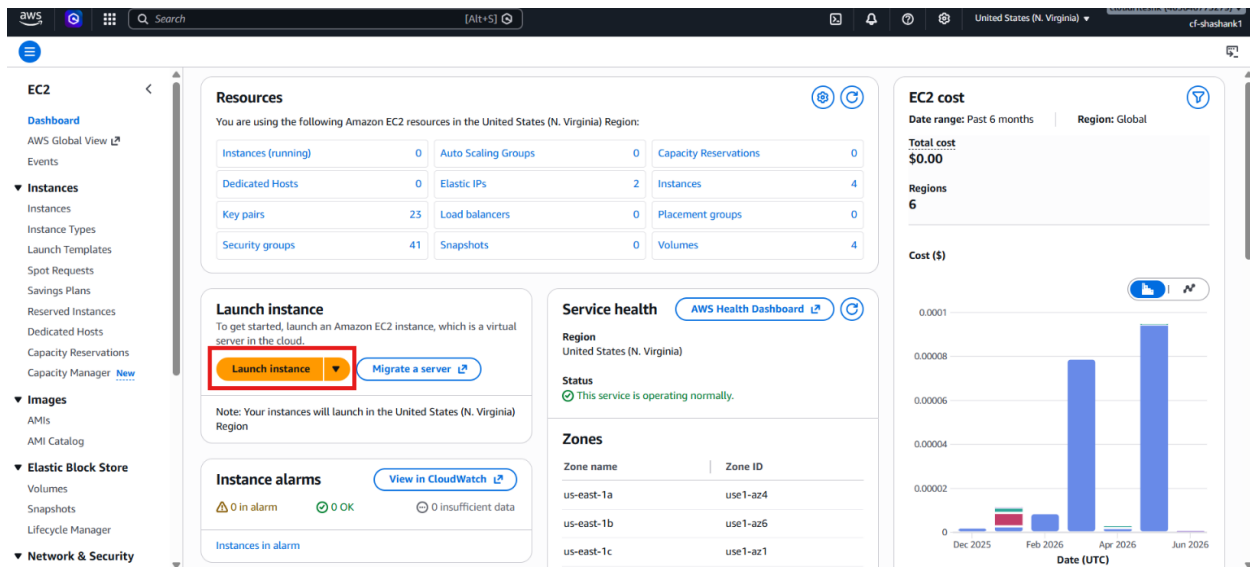
The screenshot shows the AWS IAM console for the user **S3AccessUser**. The left sidebar contains navigation links for Identity and Access Management (IAM), Access Management, and Access reports. The main content area shows the user's summary, including their ARN, console access status, and access keys. The **Permissions** tab is active, displaying a table of permissions policies. The **Add permissions** button is highlighted with a red box, and a dropdown menu is open, showing options to add permissions or create an inline policy.

- Choose **Attach policies directly**.
- Select **S3PutObjectPermission**.
- Add the permission.

The screenshot shows the 'Add permissions' page in the AWS IAM console. The 'Attach policies directly' option is selected under 'Permissions options'. Below, a table of permissions policies is displayed, with the **S3PutObjectPermission** policy highlighted by a red box. The 'Next' button at the bottom right is also highlighted with a red box.

Step 5 – Launch the EC2 Instance for the Insecure Version

- Open **Amazon EC2**.
- Click **Launch instance**.



- Enter the instance name: MyAppServer and choose the subnet and VPC.
- Select **Amazon Linux** and choose **t3.micro** instance and a **Key pair**.

Name
MyAppServer

Application and OS Images (Amazon Machine Image)

Search our full catalog including 1000s of application and OS images

Recents | **My AMIs** | **Quick Start**

Amazon Linux

Amazon Machine Image (AMI)

Amazon Linux 2023 kernel-6.18 AMI
ami-08f44e6ca90955668 (64-bit (x86), uefi-preferred) / ami-0bba2fc7fcaad71a7 (64-bit (Arm), uefi)
Virtualization: hvm ENA enabled: true Root device type: ebs

Description

Amazon Linux 2023 (kernel-6.18) is a modern, general purpose Linux-based OS that will be supported until June 2029. It is optimized for AWS and designed to provide a secure, stable and high-performance execution environment to develop and run your cloud applications.

Amazon Linux 2023 AMI 2023.12.20260622.0 x86_64 HVM kernel-6.18

Summary

Number of instances: 1

Virtual server type (instance type)
t3.micro

Firewall (security group)
New security group

Storage (volumes)
1 volume(s) - 8 GiB

Free tier: In your first year of opening an AWS account, you get 750 hours per month of t2.micro instance usage (or t3.micro where t2.micro isn't available) when used with free tier AMIs, 750 hours per month of public IPv4 address usage, 30 GiB of EBS storage, 2 million I/Os, 1 GB of snapshots, and 100 GB of bandwidth to the internet. Data transfer charges are not included as part of the free tier allowance. Charges may apply depending on your account's free tier status.

Launch instance

- Allow HTTP traffic on port **80**.

- Use the provided user data script with hardcoded credentials and do change the credentials by your own credentials.
- Scroll down and click on **Advanced Details** → **User data** and paste the script.

```
# Flask application code with embedded AWS credentials (Not Recommended)
cat << EOF > app.py
from flask import Flask, request, render_template_string
import boto3
import uuid

app = Flask(__name__)

# Replace these with your actual S3 bucket name and AWS credentials
S3_BUCKET = 'your-bucket-name'
AWS_ACCESS_KEY_ID = 'your-access-key-id'
AWS_SECRET_ACCESS_KEY = 'your-secret-access-key'

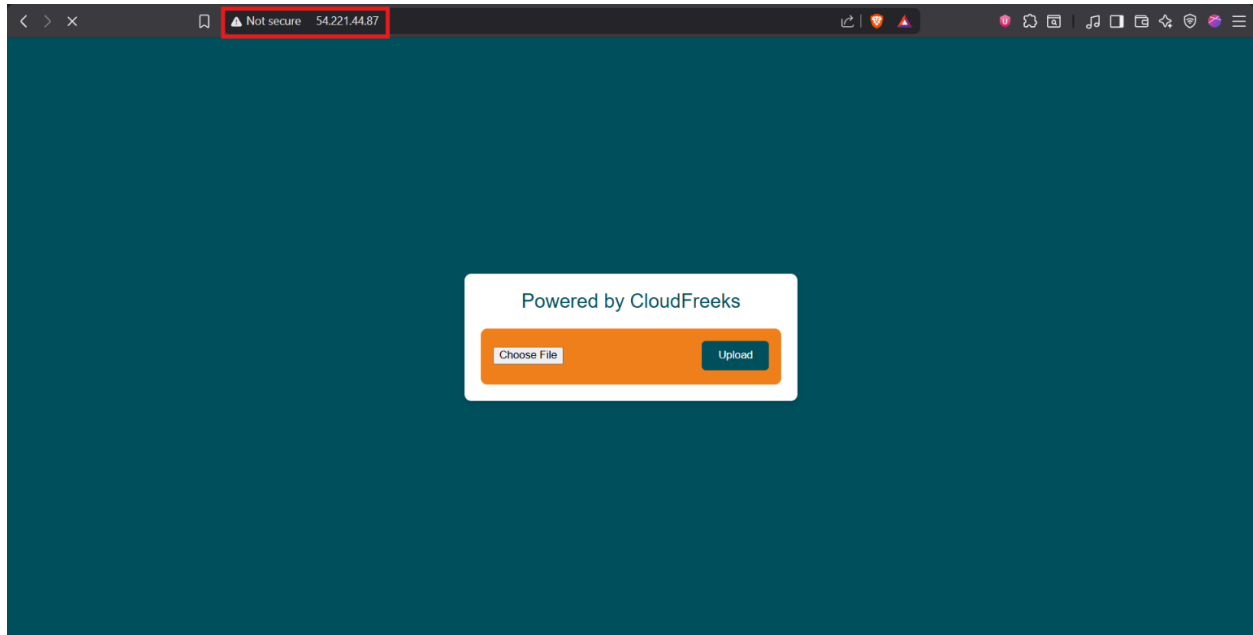
s3 = boto3.client('s3', aws_access_key_id=AWS_ACCESS_KEY_ID,
                  aws_secret_access_key=AWS_SECRET_ACCESS_KEY)
```

- Launch the instance.

Step 6 – Deploy the Flask App Using Access Keys

- Use the user data script that installs:
 - Python
 - pip
 - Flask
 - Gunicorn
 - Boto3
- Create the Flask app under /var/www.

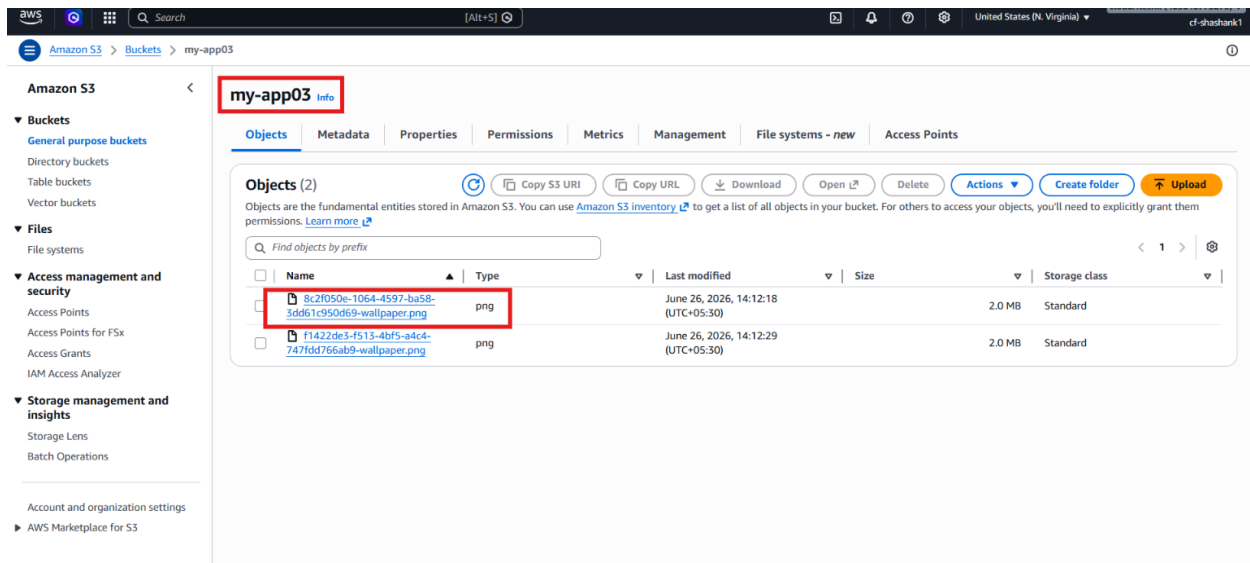
- Configure the app to use:
 - the S3 bucket name
 - the IAM user access key
 - the IAM user secret key
- Start the systemd service.
- Open the EC2 public IP in a browser.



- Upload a file through the app.



- Verify that the file appears in the S3 bucket.

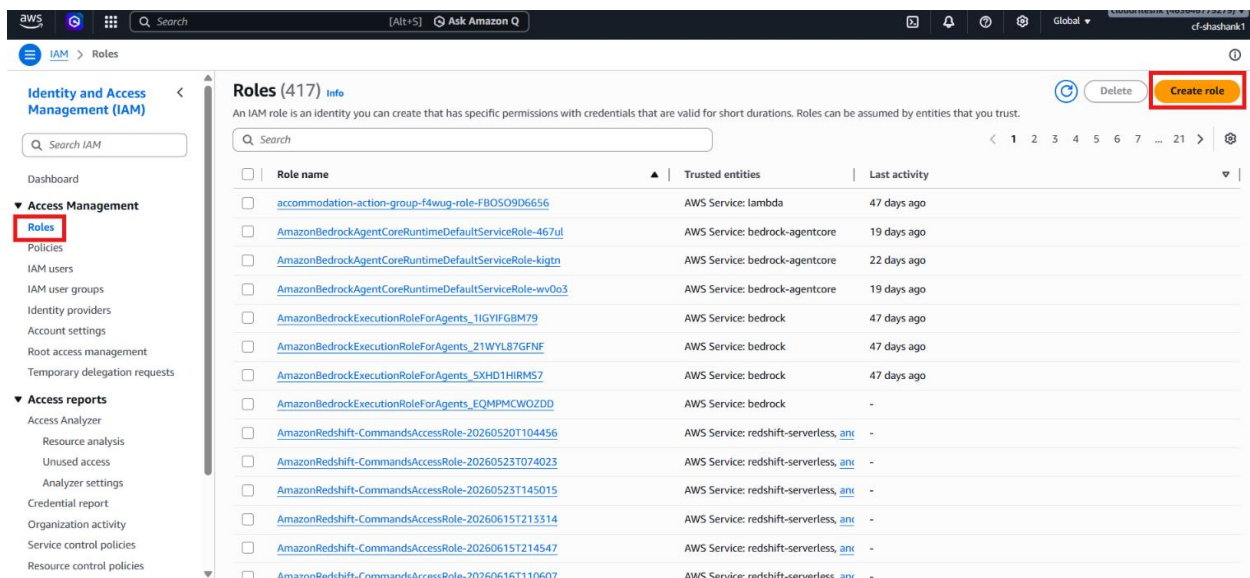


Step 7 – Observe Why the First Method Is Not Recommended

- Review the Flask code.
- Notice that the access key and secret key are embedded directly in the app.
- Understand that this creates a security risk because anyone with server access can steal the credentials.
- Treat this method only as a demonstration of the insecure pattern.

Step 8 – Create the IAM Role for the Secure Version

- Open IAM → Roles.
- Click Create role.



- Choose AWS service.
- Select EC2 as the trusted service.

Step 1 **Select trusted entity** [Info](#)

Step 2 Add permissions

Step 3 Name, review, and create

Trusted entity type

☒ **AWS service**
Allow users federated with AWS services like EC2, Lambda, or others to perform actions in this account.

☐ **AWS account**
Allow entities in other AWS accounts belonging to you or a 3rd party to perform actions in this account.

☐ **Web identity**
Allows users federated by the specified external web identity provider to assume this role to perform actions in this account.

☐ **SAML 2.0 federation**
Allow users federated with SAML 2.0 from a corporate directory to perform actions in this account.

☐ **Custom trust policy**
Create a custom trust policy to enable others to perform actions in this account.

Use case

Allow an AWS service like EC2, Lambda, or others to perform actions in this account.

Service or use case
EC2

Choose a use case for the specified service.

Use case

☒ **EC2**
Allows EC2 instances to call AWS services on your behalf.

☐ **EC2 Role for AWS Systems Manager**
Allows EC2 instances to call AWS services like CloudWatch and Systems Manager on your behalf.

☐ **EC2 Spot Fleet Role**

- Attach the same S3 upload policy.

Step 1 Select trusted entity

Step 2 **Add permissions**

Step 3 Name, review, and create

Add permissions

Permissions policies (1/2299) [Info](#)

Choose one or more policies to attach to your new role.

Filter by Type: All types 1 match

Policy name	Type	Description
☒ S3PutObjectPermission	Customer managed	-

► Set permissions boundary - optional

Cancel Previous **Next**

- Enter the role name: MyAppS3Access
- Create the role.

Step 1: Select trusted entities

Name, review, and create

Role details

Role name
Enter a meaningful name to identify this role.
MyS3AppAccess

Description
Add a short explanation for this role.
Allows EC2 instances to call AWS services on your behalf.

Trust policy

```

1 {
2   "Version": "2012-10-17",
3   "Statement": [
4     {
5       "Effect": "Allow",
6       "Action": [
7         "sts:AssumeRole"
8       ],
9       "Principal": {
10        "Service": [
11          "ec2.amazonaws.com"
12        ]
13      }
14    ]
15  }

```

Step 9 – Launch the EC2 Instance with the IAM Role

- Launch a new EC2 instance.
- Keep the same OS and instance type.
- Add port **80** for the web application.
- Attach the **MyAppS3Access** IAM role to the instance.

EC2 > Instances > Launch an instance

Advanced details

Domain join directory: Select

IAM instance profile: **MyS3AppAccess**

Hostname type: IP name

DNS Hostname: Enable IP name IPv4 (A record) DNS requests

Instance auto-recovery: Select

Shutdown behavior: Stop

Stop - Hibernate behavior: Select

Termination protection: Info

Summary

Number of instances: 1

Software Image (AMI): Amazon Linux 2023 AMI 2023.12....read more

Virtual server type (instance type): t3.micro

Firewall (security group): New security group

Storage (volumes): 1 volume(s) - 8 GiB

Free tier: In your first year of opening an AWS account, you get 750 hours per month of t2.micro instance usage (or t3.micro where t2.micro isn't available) when used with free tier AMIs, 750 hours per month of public IPv4 address usage, 30 GiB of EBS storage, 2 million I/Os, 1 GiB of snapshots, and 100 GiB of bandwidth to the internet. Data transfer charges are not included as

Launch instance

- Use the secure user data script.
- Launch the instance.

EC2 > Instances > Launch an instance

Applications or agents that use V1 for instance metadata access will break.

Metadata response hop limit [Info](#)
2

Allow tags in metadata [Info](#)
Select

User data - optional [Info](#)
Upload a file with your user data or enter it in the field.

[Choose file](#)

```
# flask application code without AWS credentials, using user role
cat << 'EOF' > app.py
from flask import Flask, request, render_template_string
import boto3
import uuid

app = Flask(__name__)

# Replace this with your actual S3 bucket name
S3_BUCKET = 'my-app03'

# Initialize the S3 client to use IAM role credentials
s3 = boto3.client('s3')

@app.route('/')
def upload_form():
```

☐ User data has already been base64 encoded

Summary

Number of instances [Info](#)
1

Software Image (AMI)
Amazon Linux 2023 AMI 2023.12...[read more](#)
ami-08f44e8eca9095668

Virtual server type (instance type)
t3.micro

Firewall (security group)
New security group

Storage (volumes)
1 volume(s) - 8 GiB

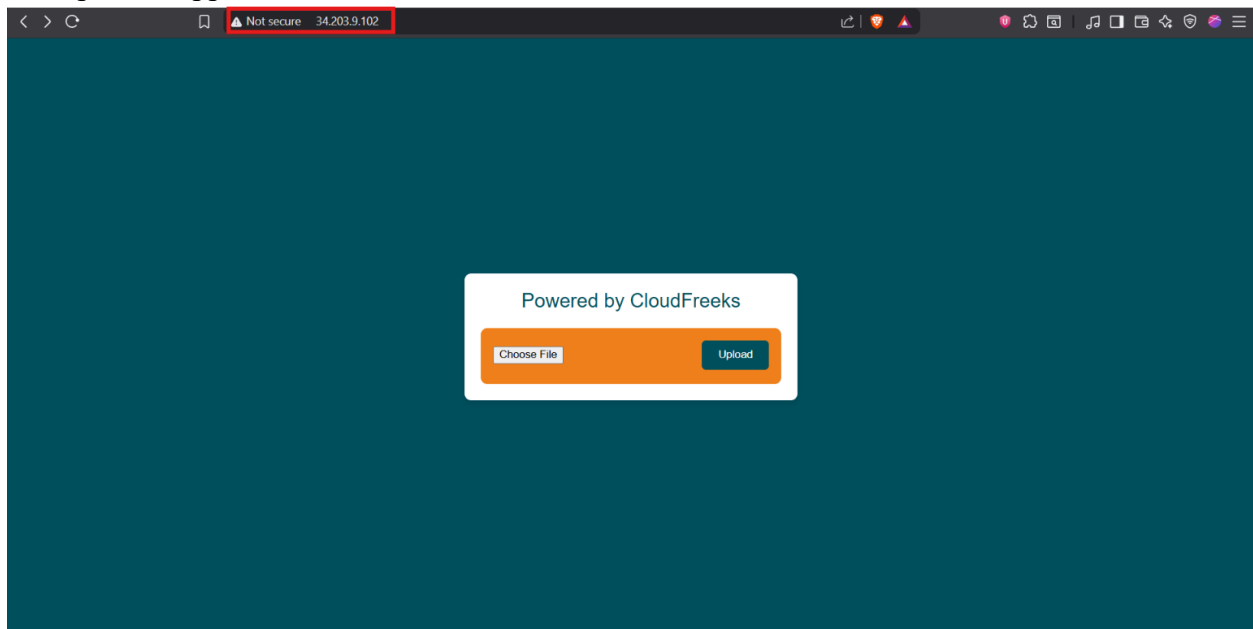
Free tier: In your first year of opening an AWS account, you get 750 hours per month of t2.micro instance usage (or t3.micro where t2.micro isn't available) when used with free tier AMIs, 750 hours per month of public IPv4 address usage, 30 GiB of EBS storage, 2 million I/Os, 1 GB of snapshots, and 100 GB of bandwidth to the internet. Data transfer charges are not included as part of the free tier allowance. Charges may apply depending on your account's free tier status.

[Cancel](#) [Launch instance](#) [Preview code](#)

- Wait for the instance to become ready.

Step 10 – Deploy the Secure Flask App

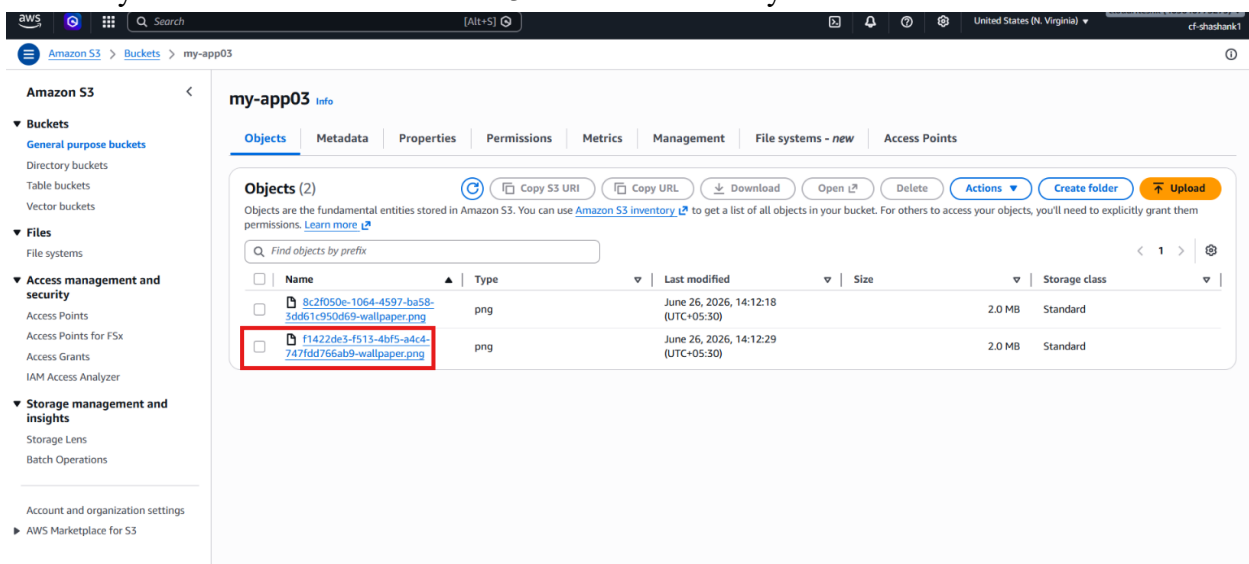
- Use the user data script that does **not** contain access keys.
- Keep only the bucket name in the code.
- Open app by opening the **Public IPv4 Address** of instance.
- Open the application in a browser.



- Upload a file again.



- Verify that the file is stored in the S3 bucket successfully.



Step 11 – Compare the Two Approaches

- Compare the insecure script and the secure script.
- Confirm that the insecure script stores AWS keys in code.
- Confirm that the secure script uses the IAM role instead.
- Verify that both methods can upload files, but only the role-based method follows AWS best practices.

Verification

- The S3 bucket was created successfully.
- The IAM user and S3 upload policy were created successfully.

- The insecure Flask app uploaded a file using embedded credentials.
- The IAM role was created for EC2.
- The secure Flask app uploaded a file without hardcoded AWS credentials.
- The uploaded file appeared in the S3 bucket.
- The role-based setup matched the recommended AWS security practice